

Axioms and Computation

Lecture 24

Section 1

Part I: Beyond CIC

Where we are

Everything in Lectures 18–23 used only the rules of CIC: dependent function types, inductive types, a universe hierarchy, and proof-irrelevant Prop.

Lean's core library (not Mathlib — the library that ships with Lean itself) extends CIC with three **axioms**. These are declared using the `axiom` keyword, which tells the kernel: “accept this constant without a definition.” Unlike `def` or `theorem`, no term is provided — the kernel takes the type on faith.

This lecture asks: what do we gain from these extensions, what do we lose, and why are they consistent?

The three extensions

- ❶ **Propositional extensionality** (`propext`): logically equivalent propositions are equal.
- ❷ **Quotient types** (`Quot.sound`): a built-in construction for quotienting a type by a relation.
- ❸ **Classical choice** (`Classical.choice`): produce data from an existence proof.

Function extensionality (`funext`) is *not* a separate axiom — it is a theorem proved from quotients. You can check axiom dependencies with `#print axioms myTheorem`.

Section 2

Part II: Propositional Extensionality

Why we might want equivalent propositions to be equal

Mathematicians do not normally say two propositions are equal -- they say equivalent.' So what does $p = q$ mean for propositions?

If we think of propositions as truth values (there are only two: true and false), then equivalent propositions *are* the same truth value. This is the set-theoretic view: each element of `Prop` is either $\emptyset$ (false) or $\{*\}$ (true), and two propositions with the same truth value are literally the same set.

CIC does not adopt this view. In CIC, $p \wedge \text{True}$ and p are different types with different constructors. Proof irrelevance says all proofs *of* p are equal, but it says nothing about whether p and q are equal *as types*. `propext` adds this collapse.

The axiom

propext identifies equivalent propositions:

```
axiom propext {a b : Prop} : (a ↔ b) → a = b
```

Example:

```
theorem tteq : (True ∧ True) = True :=  
  propext {fun ⟨h, _⟩ => h, fun h => ⟨h, h⟩}
```

The argument to propext is a proof of $(\text{True} \wedge \text{True}) \leftrightarrow \text{True}$. The anonymous constructor $\langle \dots \rangle$ builds the iff from its two directions: $\text{fun } \langle h, _ \rangle \Rightarrow h$ proves left-to-right (extract the first component); $\text{fun } h \Rightarrow \langle h, h \rangle$ proves right-to-left (duplicate into a pair).

propext does not fire automatically. You must apply it explicitly. Writing $a \leftrightarrow b$ does not give you $a = b$ — you get $a = b$ only when you invoke propext on the iff.

Application: extensional sets

Define $\text{Set } \alpha := \alpha \rightarrow \text{Prop}$ — a set of elements of α is a predicate on α . (This does *not* create Russell-paradox issues: $\text{Set } \alpha : \text{Type}$ when $\alpha : \text{Type}$, so $\text{Set } \alpha$ and α live at different universe levels. You cannot form “the set of all sets” any more than you can write $\text{Type} : \text{Type}$.)

A theory of sets is **extensional** if two sets with the same elements are equal (the word means: determined by what they contain, not how they are defined). With `propext`, we can prove this:

```
def Set ( $\alpha : \text{Type}$ ) :=  $\alpha \rightarrow \text{Prop}$ 
```

```
theorem Set.ext {a b : Set  $\alpha$ }  
  (h :  $\forall x, a\ x \leftrightarrow b\ x$ ) : a = b :=  
  funext (fun x => propext (h x))
```


Section 3

Part III: Function Extensionality

The principle

Two functions that agree on all inputs are equal:

```
theorem funext {f g :  $\alpha \rightarrow \beta$ }  
  (h :  $\forall x, f\ x = g\ x$ ) : f = g
```

This is a **theorem**, not an axiom. It is proved from the quotient construction (Part IV). If you run `#print axioms funext`, it shows `Quot.sound` — the axiom it ultimately depends on.

The word **extensional** here means the same as for sets: functions are determined by their behavior (what they map each input to), not by their definition (what code implements them). Two sorting algorithms that produce the same output for every input are the same function.

Section 4

Part IV: Quotient Types

What quotients are

Given a type α and a relation r , the **quotient** $\text{Quot } r$ is a new type where elements related by r are identified. For example, $\mathbb{Z}/7\mathbb{Z}$ is the integers where numbers differing by a multiple of 7 become equal.

Lean builds quotients into the kernel as **primitive constants** with **computation rules**. This is different from `propext`, which is an opaque axiom with no reduction. The quotient constants come with a reduction rule: $\text{Quot.lift } f \ h \ (\text{Quot.mk } r \ a)$ reduces to $f \ a$. The kernel knows how to compute with quotients.

What blocks reduction is only `Quot.sound` (the fifth constant, the actual axiom), which produces an equality — and equalities used as casts are opaque.

The construction (1/2)

```
-- The quotient type itself
axiom Quot { $\alpha$  : Sort u}
      (r :  $\alpha \rightarrow \alpha \rightarrow$  Prop) : Sort u

-- Inject a into its equivalence class
axiom Quot.mk (r :  $\alpha \rightarrow \alpha \rightarrow$  Prop)
      (a :  $\alpha$ ) : Quot r

-- Induction: every q : Quot r equals Quot.mk r a
--   for some a
axiom Quot.ind { $\beta$  : Quot r  $\rightarrow$  Prop}
      (h :  $\forall$  a,  $\beta$  (Quot.mk r a))
      (q : Quot r) :  $\beta$  q
```

The construction (2/2)

```
-- Define functions on the quotient:
-- provide f on  $\alpha$  and a proof that f respects r
axiom Quot.lift (f :  $\alpha \rightarrow \beta$ )
  (h :  $\forall a\ b, r\ a\ b \rightarrow f\ a = f\ b$ )
  : Quot r  $\rightarrow \beta$ 
```

Quot.lift is how you define functions on the quotient: provide a function on α and a proof that it maps r -related elements to the same result. The kernel installs $\text{Quot.lift } f\ h\ (\text{Quot.mk } r\ a) = f\ a$ as a reduction rule.

Why these four are not enough

The four constants above are very weak. You could satisfy them by setting $\text{Quot } r = \alpha$, making Quot.mk the identity, and having $\text{Quot.lift } f \ h = f$. All the types check out. But this does not actually quotient anything — r -related elements remain distinct.

Quot.sound: the real axiom

The genuine axiom is:

```
axiom Quot.sound {a b :  $\alpha$ }  
      : r a b  $\rightarrow$  Quot.mk r a = Quot.mk r b
```

This says: if $r\ a\ b$, then a and b become equal in the quotient. The trivial model $\text{Quot } r = \alpha$ cannot satisfy this (it would require $r\ a\ b \rightarrow a = b$ for every r). So `Quot.sound` is the genuine axiom — the one that `#print axioms` will report.

Can a “bad” relation make this inconsistent? No. `Quot.sound` is a one-way door: $r\ a\ b$ implies $\text{Quot.mk } r\ a = \text{Quot.mk } r\ b$, but *not* the reverse. Equalities in the quotient do not propagate back to α . Even if r relates everything to everything, the quotient collapses to one element, but α is untouched.

Example: integers as a quotient

Represent the integer $a - b$ as the pair $(a, b) : \text{Nat} \times \text{Nat}$. Two pairs represent the same integer when $a + d = b + c$:

```
def intRel (p q : Nat × Nat) : Prop :=  
  p.1 + q.2 = p.2 + q.1  
  
def MyInt := Quot intRel  
  
-- (3,1) and (5,3) both represent 2:  
example : Quot.mk intRel (3, 1)  
  = Quot.mk intRel (5, 3) :=  
  Quot.sound (show 3 + 3 = 1 + 5 by omega)
```

How funext follows from quotients

The proof defines “agree pointwise” as an equivalence relation on functions, forms the quotient, and uses `Quot.lift` (which has a computation rule) to show that extensionally equal functions are equal. The proof goes through `Quot.sound`, which is why `#print axioms funext` shows it.

Since `funext` is proved from `Quot.sound` (an opaque axiom), using `funext` in a cast will block `#reduce` — just like `propext`. But it is compatible with the compiler (`#eval`), because the equality it produces lives in `Prop` and gets erased.

What blocks computation

```
def f (x : Nat) := x
def g (x : Nat) := 0 + x
theorem f_eq_g : f = g :=
  funext fun x => (Nat.zero_add x).symm

#check f_eq_g    -- f = g
#reduce f_eq_g    -- stuck: cannot reduce through Quot.sound
```

`f_eq_g` uses `funext`, which depends on the opaque axiom `Quot.sound`. The kernel's reducer cannot simplify through it. This does not affect compiled code: `#eval f 5` and `#eval g 5` both return 5. Propositional content is erased at compile time.

Only `Classical.choice` produces data from `Prop`, requiring noncomputable.

Section 5

Part V: Classical Choice

The axiom

```
axiom Classical.choice { $\alpha$  : Sort u}  
      : Nonempty  $\alpha$   $\rightarrow$   $\alpha$ 
```

Nonempty α is a Prop: you know α has an element, but the proof carries no information about which one (proof irrelevance). Classical.choice takes this bare assertion and produces an actual element. There is no algorithm to do this.

How is this different from $\exists$?

In constructive Lean, $\exists x, P x$ does carry a witness — you build it with $\langle w, h \rangle$. But $\exists$ lives in `Prop`. By the large elimination rule (Lecture 23), you can only extract the witness into another `Prop`, not into `Type`.

`Classical.choice` bypasses this restriction: it produces data (in `Type`) from a proof (in `Prop`). Any definition that does this must be marked `noncomputable`.

Why is choice controversial?

The axiom of choice has been debated since Zermelo introduced it in 1904. The objection: you assert a selection function exists without saying *how* to select. In type theory, this is the Prop/Type boundary violation: CIC is designed so that proofs (Prop) and programs (Type) live in separate worlds. Choice breaks the wall.

Despite this, choice is used throughout mathematics. In Lean, you can use it freely — the `noncomputable` tag makes the cost explicit.

Why does choice not cause inconsistency?

In Lecture 15 we saw Girard's paradox: $\text{Type} : \text{Type}$ is inconsistent because it allows encoding a self-referential well-ordering argument. Choice does not do this — it does not introduce self-reference or collapse the universe hierarchy.

The consistency argument is **relative**: if ZFC with inaccessible cardinals is consistent (which is the background assumption for Lean's universe hierarchy), then CIC plus these three axioms is also consistent. This is shown by constructing a set-theoretic model where all three hold and False has no inhabitant. (See Werner, *Sets in Types, Types in Sets*, '' 1997; and Carneiro, *The Type Theory of Lean*,'' 2019, for Lean specifically.)

We cannot prove consistency *absolutely* (Gödel's second incompleteness theorem). But relative consistency is the standard for all of mathematics. Consistency is also different from computational content: choice is consistent but non-constructive.

What is the point of noncomputable definitions?

```
noncomputable def choose_prime (n : Nat)
  (h :  $\exists p, p > n \wedge \text{Nat.Prime } p$ ) : Nat :=
  Classical.choose h
```

You might ask: h was built with $\langle w, hw \rangle$, so the witness w is right there — why can't we just extract it? Because $\exists$ lives in Prop, and the large elimination rule (Lecture 23) forbids extracting data in Type from a proof in Prop. Writing `h.1` to get a Nat does not typecheck.

`Classical.choose` bypasses this restriction via `Classical.choice`. The result lets you say “pick a prime p larger than n ” without specifying how to find it. You can then use `Classical.choose_spec h` (a proof that the chosen value is prime and $> n$) in subsequent theorems. The classical mathematician's workflow: name objects, prove things about them, never compute them.

Section 6

Part VI: Diaconescu's Theorem

Choice implies excluded middle

`propext`, `Quot.sound` (which gives `funext`), and `Classical.choice` together imply the law of the excluded middle: $\forall (p : \text{Prop}), p \vee \neg p$.

This is Diaconescu's theorem (1975). Note that $p \vee \neg p$ is a **theorem** here, not an axiom — it is *derived*. `#print axioms Classical.em` shows `propext`, `Quot.sound`, and `Classical.choice`.

Given a proposition p , define two **sets of propositions**:

$$U = \{x : \text{Prop} \mid x = \text{True} \vee p\}$$

$$V = \{x : \text{Prop} \mid x = \text{False} \vee p\}$$

U contains `True` (and everything else if p holds). V contains `False` (and everything else if p holds). Now define:

$$u := \text{Choice } U \quad v := \text{Choice } V$$

We know $u \in U$ and $v \in V$, but not which propositions u and v are.

Why Classical.choice?

When p holds, every proposition is in both U and V (via $\vee$ -right), so by propext and funext , $U = V$. Since $u = \text{Choice } U$ and $v = \text{Choice } V$, we can rewrite U to V in the first equation:

$$u = \text{Choice } U = \text{Choice } V = v$$

After the rewrite, u and v are syntactically the same term, so $u = v$. That is the whole argument: $p \rightarrow U = V \rightarrow u = v$.

The other direction

We also know $U(u)$ and $V(v)$. Each is a disjunction, giving four cases:

$U(u)$	$V(v)$	Conclusion
$u = \text{True}$	$v = \text{False}$	$u \neq v$
$u = \text{True}$	p	p
p	$v = \text{False}$	p
p	p	p

In three cases, p holds. In the fourth, $u \neq v$. So: $u \neq v \vee p$.

Putting it together

We have:

- $p \rightarrow u = v$ (from the key property of choice)
- $u \neq v \vee p$ (from the case analysis)

The contrapositive of the first is $u \neq v \rightarrow \neg p$. Matching on the second: if $u \neq v$, then $\neg p$; if p , then p . Either way, $p \vee \neg p$.

Consequences

From `Classical.em`, Lean derives double negation elimination. More interesting is:

```
-- Recall (Lecture 22):  
-- class inductive Decidable (p : Prop) where  
--   | isFalse (h : ¬p) : Decidable p  
--   | isTrue  (h : p)  : Decidable p  
  
noncomputable instance Classical.propDecidable  
  (p : Prop) : Decidable p :=  
  choice <| match em p with  
  | Or.inl h  => { .isTrue h }  
  | Or.inr h  => { .isFalse h }
```


Reading the proof

Why not just match on `em p` directly and return `.isTrue h` or `.isFalse h`? Because `em p` has type $p \vee \neg p$, and $\vee$ has two constructors — large elimination (Lecture 23) forbids matching on it to produce data in `Type`.

The trick: first match $p \vee \neg p$ to produce `Nonempty (Decidable p)` — this is `Prop → Prop`, so small elimination is fine. Then use `Classical.choice` to extract the `Decidable p` from the `Nonempty` wrapper.

Every proposition is now decidable, but noncomputable: there is no algorithm. This instance has low priority (Lecture 22), so Lean prefers constructive instances when available.

Section 7

Part VII: The Spectrum

Three levels

Pure: no axioms. Closed Nat terms reduce to numerals. Lectures 18–21.

Compatible with compilation: uses `propext`, `Quot.sound`, or `Classical.em`. The kernel's `#reduce` gets stuck but the compiler (`#eval`) works — it erases Prop content. Most practical Lean code lives here.

Noncomputable: uses `Classical.choice` to produce data. Cannot be compiled.

The boundary in practice

```
-- Pure: no axioms
def factorial : Nat → Nat
  | 0 => 1
  | n + 1 => (n + 1) * factorial n

-- Compatible: uses Decidable (a type class)
def abs (x : Int) : Int :=
  if x ≥ 0 then x else -x

-- Noncomputable: uses Classical.choice
noncomputable def choose_prime (n : Nat)
  (h : ∃ p, p > n ∧ Nat.Prime p) : Nat :=
  Classical.choose h
```

Section 8

Part VIII: Why Can We Trust Lean?

The question from Lecture 1

In the first lecture we asked: when a proof assistant says a proof is correct, why should we believe it?

We now have the tools to answer. Lean's kernel does not search for proofs or do anything clever. It **type-checks**: given a term t and a type T , it verifies that $t : T$ according to the rules of CIC. By Curry-Howard (Lectures 5–6), this is the same as verifying that t is a proof of proposition T .

What the kernel checks

The kernel implements a small number of rules:

- **Dependent function types** (Π -types) from the λ -cube (Lecture 14).
- **Inductive types** with their eliminators (Lectures 17–18).
- **A universe hierarchy** $\text{Prop} : \text{Type}_0 : \text{Type}_1 : \dots$ avoiding Girard's paradox (Lecture 15).
- **Computation rules:** β -reduction (Lecture 1), δ -reduction (unfolding definitions), ι -reduction (pattern matching).

Everything else — tactics, type class resolution, elaboration, `termination_by` — is *elaboration*: it constructs terms that the kernel then checks. If the elaborator has a bug, the kernel catches it.

Why the rules are sound

The rules of CIC are well-studied mathematically. The three axioms (`propext`, `Quot.sound`, `Classical.choice`) have been shown consistent relative to ZFC with inaccessible cardinals: there exist set-theoretic models where all three hold and `False` is unprovable.

The kernel itself is roughly 10,000 lines of code. Everything outside the kernel — the tactic framework, the simplifier, `omega`, `simp`, the compiler — could be replaced entirely without affecting the trustworthiness of proofs. This is the **de Bruijn criterion**: the trusted base is small enough to audit.

So when Lean says a proof is correct, it means: the kernel verified that a specific term has a specific type, using rules whose soundness rests on the mathematics we developed in this course.